ATELIER 3 Le harness

Algorithmie comparative - J2 - Lundi 9 novembre 2026, 11h50 - 50 minutes, en binôme

La question à laquelle l'atelier répond

Le benchmark du CTO dit que la nouvelle variante est des dizaines de fois plus lente : que vaut cette mesure, et comment mesurer pour que le chiffre ne mente pas ?

Ce que vous produisez : bench/BIAIS-bench-naif.md, votre harness complété (bench/harness.ts, il sert jusqu'au projet).

Le rendu attendu en fin d'atelier

Le correcteur de l'atelier répond sans KO :

```
OK  Les biais du benchmark du CTO sont écrits
OK  À COMPLÉTER 1 : l'échauffement est fait avant les mesures
OK  À COMPLÉTER 2 : la mémoire est relevée
OK  À COMPLÉTER 3 : le résultat est consommé par la somme de contrôle
OK  À COMPLÉTER 4 : le résumé donne la médiane
OK  À COMPLÉTER 5 : les mesures sont exportées en CSV
OK  L'affectation est mesurée avec 2 tours d'échauffement
OK  L'affectation est mesurée avec un échauffement réglé, et les variantes concordent
8 contrôles, 0 KO
```

Pour démarrer

Le premier sous-objectif se fait ensemble, avec l'intervenant, pendant les dix premières minutes. Faites le premier sous-objectif avec l'intervenant, puis lisez la page 2.

Sous-objectif 1, démonter le benchmark du CTO : la première commande, à la racine du dépôt (le dossier efrei-logistique-perf).

Windows (PowerShell), macOS et Linux

```
npm run bench:naif
```

Les sous-objectifs

1. Démonter le benchmark du CTO (consigne 1 de la slide)

Pourquoi. Un benchmark a un auteur et une thèse : avant de mesurer vous-mêmes, repérez ce qui fait mentir celui-ci. On ne corrige pas son fichier.

Première action. Lancez `npm run bench:naif`, copiez le gabarit des biais, puis lisez bench/bench-naif.ts avec les huit questions, recopiées dans le gabarit.

Ce que vous devez voir. Dans bench/BIAIS-bench-naif.md, au moins trois entrées au format `- biais 1 - ligne N - question Q : ce qu'il fausse` ; "lignes N à M" et "questions P et Q" sont acceptés.

2. Compléter les cinq trous (consignes 2 et 3 de la slide)

Pourquoi. Le harness est votre banc de mesure : échauffement, mémoire, résultat consommé, médiane, export CSV.

Première action. Ouvrez bench/harness.ts, lisez le commentaire À COMPLÉTER 1 et les lignes qui l'entourent, complétez-le, puis lancez `npm run verifier -- 3` ; recommencez trou par trou.

Ce que vous devez voir. Un OK de plus à chaque trou : le correcteur branche votre harness sur un sujet témoin dont il connaît les réponses.

3. Régler l'échauffement (consignes 4 et 5 de la slide)

Pourquoi. Le nombre de tours d'échauffement se règle par la mesure, pas par une règle apprise.

Première action. Mesurez tri et map avec 2 tours d'échauffement, puis avec 200, et comparez les médianes à 1 000 et à 10 000 colis.

Ce que vous devez voir. Sur chaque ligne de mesure, la même valeur dans la colonne controle pour tri et map à une taille donnée.

4. Vérifier (consigne 6 de la slide)

Pourquoi. Le correcteur relance votre harness sur le sujet témoin : c'est lui qui dit que les cinq trous sont faits, pas vous.

Première action. Lancez `npm run verifier -- 3` ; gardez sous la main la ligne qui commence par `[harness] node=` et vos médianes à 10 000 colis : trois binômes les diront à 13h40.

Ce que vous devez voir. Le correcteur finit sur "8 contrôles, 0 KO" : les biais, chacun des cinq trous sur le témoin, les deux mesures.

> **La décision de l'atelier**
>
> Combien de tours d'échauffement retenez-vous pour l'atelier 4, et quelles médianes le justifient (tri à 1 000 et à 10 000 colis, avec 2 puis 200 tours) ? Cette ligne entre dans la rubrique 3 de l'ADR-001.

> **La question de compréhension**
>
> À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux pour la mise en commun de 13h40.

Les commandes de chaque sous-objectif, pour Windows, macOS et Linux, avec leur sortie attendue, sont dans l'annexe. À 17h00, le binôme commite : git add -A puis git commit -m "J2".

Si ça coince

- "global.gc is not a function", ou une mémoire nulle ou négative : le harness a été lancé avec npx tsx ; lancez-le par `npm run bench`.
- Le harness s'arrête sur ARRÊT : deux variantes ne donnent pas la même somme de contrôle ; une variante a été modifiée, ou le tableau d'entrée est trié en place. Revenez à la version du dépôt de bench/variantes.
- Le correcteur dit que la chauffe ne s'arrête pas : la boucle du trou 1 doit aussi s'arrêter quand la chauffe dure depuis chauffeMax millisecondes. Calculez l'heure de fin avant la boucle, comparez-la à performance.now() à chaque tour.

Toujours bloqué après ces trois pistes : levez la main, ou écrivez sur Teams. D'autres cas sont à la fin de l'annexe.

Si vous avez fini

- Mesurez les variantes dans l'ordre inverse (`--variantes map,tri`) : le classement bouge-t-il ? C'est la question 8.
- Cherchez sur votre poste le nombre de tours à partir duquel la médiane de tri à 1 000 colis cesse de baisser, avec `--variantes tri --tailles 1000` : `--echauffement` 60, 100, 150, 200, 300.

Où trouver un exemple dans le code

bench/sujets/temoin.ts montre ce qu'un sujet fournit au harness (charger, préparer, les variantes, la somme de contrôle). bench/sujets/affectation.ts fait la même chose pour l'affectation. Ni l'un ni l'autre ne contient le code des trous.

Annexe : toutes les commandes, dans l'ordre

Toutes les commandes se lancent à la racine du dépôt. Les sorties montrées sont de vraies sorties, capturées sous Windows en jouant cette fiche ; vos durées seront différentes, vos nombres de colis, de couples et vos sommes de contrôle non. "Extrait" signale qu'on n'en montre que les lignes utiles.

1. Démonter le benchmark du CTO

Windows (PowerShell), macOS et Linux

```
npm run bench:naif
```

Vos durées seront différentes, la conclusion non : elle est écrite dans le code.

Ce que vous devez voir (sortie réelle, Windows) :

```
Benchmark d'affectation sur 300 colis (grande)
  map (nouvelle)      164 ms
  tri                   2 ms
  naive (legacy)        5 ms
La nouvelle variante est 33 fois plus lente que le legacy.
Conclusion : on garde le code actuel.
```

Windows (PowerShell)

```
Copy-Item bench\BIAIS-bench-naif-gabarit.md bench\BIAIS-bench-naif.md
code bench\bench-naif.ts
```

macOS et Linux

```
cp bench/BIAIS-bench-naif-gabarit.md bench/BIAIS-bench-naif.md
code bench/bench-naif.ts
```

2. Compléter les cinq trous

Windows (PowerShell)

```
code bench\harness.ts
```

macOS et Linux

```
code bench/harness.ts
```

Inutile de lire les 147 lignes : cherchez À COMPLÉTER, le reste du harness est écrit.

Windows (PowerShell), macOS et Linux

```
npm run verifier -- 3
```

Après chaque trou. La sortie complète, huit OK, est au sous-objectif 4.

Windows (PowerShell), macOS et Linux

```
npm run bench -- --sujet temoin --repetitions 5 --echauffement 3
```

Facultatif, une fois les cinq trous faits : le résumé que le correcteur lit sur le témoin. Lancez toujours le harness par npm run bench : le script ajoute l'option --expose-gc, sans laquelle global.gc n'existe pas.

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
variante        taille   duree_ms   memoire_ko
reguliere           10       2.01           30
reguliere           20       2.01            1
irreguliere         10       2.01            1
irreguliere         20       2.01            1
gourmande           10       0.34          416
gourmande           20       0.10          313
[temoin] appels=48
```

3. Régler l'échauffement

Windows (PowerShell), macOS et Linux

```
npm run bench -- --variantes tri,map --repetitions 5 --echauffement 2
```

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
[harness] node=v24.14.1 processeur="12th Gen Intel(R) Core(TM) i9-12900K" memoire=128Go
systeme="Windows_NT 10.0.26300"
[harness] variante=tri taille=10000 repetition=1 duree_ms=16.85 memoire_ko=1827
controle=058848ce
[harness] variante=map taille=10000 repetition=1 duree_ms=6.10 memoire_ko=5215
controle=058848ce
variante        taille   duree_ms   memoire_ko
tri               1000       4.26         1498
tri               2000       6.95        10559
tri               5000      11.83        11004
tri              10000      16.80         1830
map               1000       0.91         1033
map               2000       1.98         2420
map               5000       3.62         7822
map              10000       6.10         5215
```

Windows (PowerShell), macOS et Linux

```
npm run bench -- --variantes tri,map --repetitions 5 --echauffement 200
```

Moins d'une minute : le harness arrête la chauffe au bout de 5 secondes. La colonne memoire_ko varie fortement d'un poste à l'autre et d'un lancement à l'autre : tri à 10 000 colis relève 1,8 Mo avec 2 tours et 33,5 Mo avec 200 sur le poste de capture, 2,4 Mo sur un autre poste. Ce n'est pas une erreur du trou 2 : le correcteur le juge sur le sujet témoin.

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
[harness] node=v24.14.1 processeur="12th Gen Intel(R) Core(TM) i9-12900K" memoire=128Go
systeme="Windows_NT 10.0.26300"
[harness] variante=tri taille=10000 repetition=1 duree_ms=17.48 memoire_ko=34314
controle=058848ce
[harness] variante=map taille=10000 repetition=1 duree_ms=6.39 memoire_ko=5204
controle=058848ce
variante        taille   duree_ms   memoire_ko
tri               1000       2.54         3487
tri               2000       4.71         6949
tri               5000       9.80        17282
tri              10000      17.44        34314
map               1000       0.52          517
map               2000       0.99         1021
map               5000       2.73         2645
map              10000       6.25         5204
```

4. Vérifier

Windows (PowerShell), macOS et Linux

```
npm run verifier -- 3
```

La ligne machine est celle qui commence par [harness] node= dans la sortie du harness (sous-objectif 3) ; à 13h40, dites-la avec "sur secteur" ou "sur batterie".

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
OK  Les biais du benchmark du CTO sont écrits
OK  À COMPLÉTER 1 : l'échauffement est fait avant les mesures
OK  À COMPLÉTER 2 : la mémoire est relevée
OK  À COMPLÉTER 3 : le résultat est consommé par la somme de contrôle
OK  À COMPLÉTER 4 : le résumé donne la médiane
OK  À COMPLÉTER 5 : les mesures sont exportées en CSV
OK  L'affectation est mesurée avec 2 tours d'échauffement
OK  L'affectation est mesurée avec un échauffement réglé, et les variantes concordent
8 contrôles, 0 KO
```

Cas particuliers

- Le correcteur dit que le résumé donne la moyenne : le trou 4 n'est pas fait, ou il trie le tableau des mesures sans le copier.
- Le correcteur ne compte pas vos biais : chaque entrée commence par un tiret et cite au moins un numéro de ligne et un numéro de question, sur la même ligne.
- Le CSV s'ouvre mal dans Excel : le point-virgule et le point décimal sont voulus ; l'import se règle dans Excel.
- macOS : la commande code est introuvable. Dans VS Code, palette de commandes, "Shell Command: Install 'code' command in PATH" ; ou ouvrez le fichier dans votre éditeur.